

- Title : **Machine Learning for Signal Processing Assignment-3**
- Name : **Md Samsad Alam**
- SR No : **10-01-03-10-51-24-1-24326**
- Course : MTech AI

1 Sentiment Analysis

(a) Visualization of Semantic Clustering

Get the necessary imports

I have imported the following libraries. I have explained the reason for importing each library. All of them are basic utility libraries for data pre-processing and deep learning and do not go out of the requirement of what we are supposed to import for the assignment.

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from gensim.models import KeyedVectors #type:ignore
import gensim.downloader as gensim_api #type:ignore
import re # Cleaning and preprocessing strings
import os
from bs4 import BeautifulSoup as bs # To clean HTML tags from reviews

import numpy as np #type: ignore
import pandas as pd #type: ignore
import matplotlib.pyplot as plt #type:ignore
import nltk #type:ignore # Tokenization and vocab building
nltk.download('stopwords')
nltk.download('punkt_tab')
from nltk.corpus import stopwords #type:ignore
from tqdm import tqdm #type:ignore
```

```
SEED = 746
np.random.seed(SEED)
device_idx = 6
device = torch.device(f'cuda:{device_idx}') if torch.cuda.is_available() else
↳ torch.device('cpu')
#device = torch.device('cpu')
print('Using device: ', device)
```

Functions to do Train test split, Tokenize the data, Build list of unique words in Training set to train PCA and corresponding support functions

```
def text_to_tokens(text):
    get_stop_words = set(stopwords.words('english'))
    text = text.lower()
    tokens = re.findall(r'\w+', text)
    tokens = [token for token in tokens if token not in get_stop_words]
    return tokens

def split_data(data_path, train_size, test_size):
```

```

df = pd.read_csv(data_path)
reviews, sentiments = df.review.values.tolist(),
↪ df.sentiment.values.tolist()
# convert all sentiments to integers
for i in range(len(sentiments)):
    sentiments[i] = 0 if sentiments[i] == 'negative' else 1

random_indices = np.arange(len(reviews))
np.random.shuffle(random_indices)
train_reviews, train_sentiments = [reviews[i] for i in
↪ random_indices[:train_size]], \
                                [sentiments[i] for i in
↪ random_indices[:train_size]]
test_reviews, test_sentiments = [reviews[i] for i in
↪ random_indices[train_size:train_size + test_size]], \
                                [sentiments[i] for i in
↪ random_indices[train_size:train_
↪ size+test_size]]
val_reviews, val_sentiments = [reviews[i] for i in
↪ random_indices[train_size + test_size:]], \
                                [sentiments[i] for i in random_indic_
↪ es[train_size+test_size:]]

sentiment_data = dict(
    train_reviews = train_reviews,
    train_sentiments = train_sentiments,
    test_reviews = test_reviews,
    test_sentiments = test_sentiments,
    val_reviews = val_reviews,
    val_sentiments = val_sentiments
)

return sentiment_data

def tokenize_data(document):
    '''
    document: should be a list of strings where each strings is one sample
    '''
    tokenized_document = []
    for sentence in document:
        tokenized = text_to_tokens(sentence)
        tokenized_document.append(tokenized)
    return tokenized_document

def get_info(x, y, data_type):
    print(f'\nDetails of {data_type} data: \n')
    assert len(x) == len(y)

```

```

print(f'Number of Samples in {data_type}: {len(x)} ')
print('Number of positive sentiments: ', np.sum([1 for i in y if i==1]))
print('Number of negative sentiments: ', np.sum([1 for i in y if i==0]))

def get_unique_words(texts, cutoff_freq):
    """
    texts: should be list of strings where each str is the movie review
    cutoff_freq: minimum amount of time the word should occur for it to be
    ↪ added in vocabulary
    returns - list of unique words
    """
    if not isinstance(texts, list):
        texts = [texts]
    word_list = []
    # Process each document in texts here and store the list of all words
    for x in texts:
        filtered_tokens = text_to_tokens(x)
        word_list += filtered_tokens

    unique_words, counts = np.unique(word_list, return_counts=True)
    final_unique_words = []
    for i in range(len(unique_words)):
        if counts[i] >= cutoff_freq:
            final_unique_words.append(unique_words[i])

    return final_unique_words

```

Function to do PCA and transform data

```

def PCA_reduce(xtrain, xtest, M):
    # Normal PCA computation without any high-dim use
    # X is expected to of shape N x D where N is number of samples and D is
    ↪ dimension of each sample
    N = xtrain.shape[0]

    # Normalize the data
    X_mean = np.mean(xtrain, axis = 0, keepdims = True)
    X_normalized = xtrain - X_mean

    Cov_mat = X_normalized.T @ X_normalized / N

    # Compute the eigen values and eigen vectors
    eigvals, eigvecs = np.linalg.eigh(Cov_mat)
    eigvecs = eigvecs[:,::-1]

    # Return the transformed xtest using M top eigenvectors

```

```
return xtest @ eigvecs[:, :M]
```

Split the data and preprocess, get unique words dataset and train PCA on train set

```
path_to_data = '/data/home/samsadalam/MLSP_Assignment/Assignment3/IMDB
↳ Dataset.csv'
train_size = 40000
test_size = 5000
train_test_val_split = split_data(path_to_data, train_size, test_size)

xtrain, ytrain = train_test_val_split['train_reviews'],
↳ train_test_val_split['train_sentiments']
xtest, ytest = train_test_val_split['test_reviews'],
↳ train_test_val_split['test_sentiments']
xval, yval = train_test_val_split['val_reviews'],
↳ train_test_val_split['val_sentiments']

get_info(xtrain, ytrain, 'Train')
get_info(xtest, ytest, 'Test')
get_info(xval, yval, 'Validation')
```

Details of Train data:

Number of Samples in Train: 40000
Number of positive sentiments: 19946
Number of negative sentiments: 20054

Details of Test data:

Number of Samples in Test: 5000
Number of positive sentiments: 2519
Number of negative sentiments: 2481

Details of Validation data:

Number of Samples in Validation: 5000
Number of positive sentiments: 2535
Number of negative sentiments: 2465

```
train_unique_words = get_unique_words(xtrain, cutoff_freq = 1)
subval_unique_words = get_unique_words(xval[:10], cutoff_freq= 1)
print(f'Number of unique words in train data: {len(train_unique_words)}')
print(f'Number of unique words in sub-validation data:
↳ {len(subval_unique_words)}')
```

```
# Build X matrix from train data to compute PCA
```

```

X_train = []
for word in train_unique_words:
    if word_vectors.__contains__(word):
        X_train.append(word_vectors.__getitem__(word))

X_train = np.stack(X_train, axis = 0)

print(X_train.shape)

X_subval = []
X_subval_words = []
for word in list_of_plottable_words:
    if word_vectors.__contains__(word):
        X_subval.append(word_vectors.__getitem__(word))
        X_subval_words.append(word)

X_subval = np.stack(X_subval, axis = 0)
print(X_subval.shape)

pca_reduced_x = PCA_reduce(X_train, X_subval, M = 2)
print(pca_reduced_x.shape)

# Plot the Semantic Clustering

# Now let's do the plotting
plt.figure(figsize = (16,16))
for i in range(pca_reduced_x.shape[0]):
    word = X_subval_words[i]
    x,y = pca_reduced_x[i,0], pca_reduced_x[i,1]
    plt.scatter(x, y, color = 'blue', alpha = 0.5)
    plt.text(x, y, word, fontsize = 8)

plt.savefig('/data/home/samsadalam/MLSP_Assignment/Assignment3/q1_pca_plot.png')
plt.close()

```

Number of unique words in train data: 92875 Number of unique words in sub-validation data: 763

Set of words from Test set for visualization of Semantic Clustering

Instead of plotting all the words from some random 10 sentences from the test set, I thought it would be better to create a set of words under different grouping and then plot them to visualize clustering. This will lead to clean plotting with few words to visualize under each cluster.

Obeservations:

The plot of semantic clustering is given in Figure 1. We can see that the similar words under different category have been clustered together. For example words like cinematography, actor; movie, script, storyline etc that denote movie genre are grouped together on bottom left side of the figure. Positive sentiments like amazing, smart, wonderful, interesting etc. are also grouped together. Negative sentiments like dull, irritating, boring, unwatchable are grouped together. In this way we can conclude that Word2Vec indeed captures word semantics and their mutual relationships.

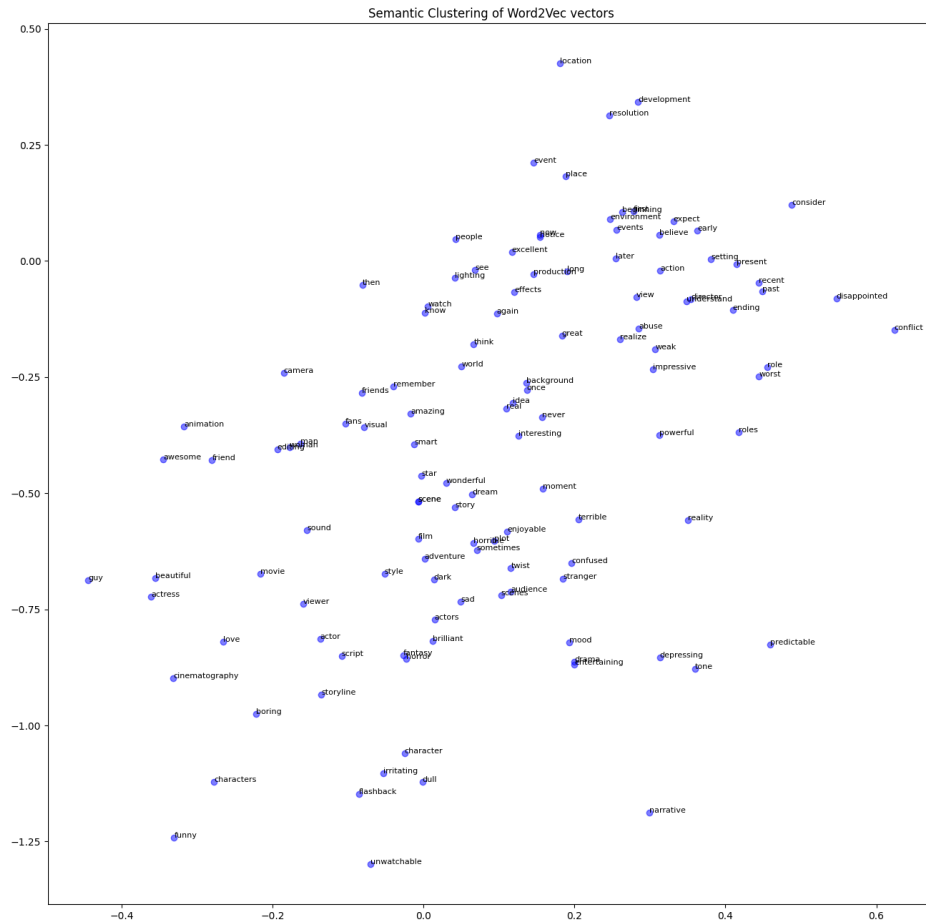


Figure 1: Semantic clustering of Word2vec vectors

For the rest of the parts of **Question-1** I have build a general code for data pre-processing, data-loading, training and evaluation. Only models change from question to question. So I will paste all the codes in one place except for the models which I will place at their corresponding questions. Please note that all such models are actually executed in `models.py` file.

(b) LSTM Model with Average Pooling

Data Pre-processing Module

```
from gensim.models import KeyedVectors #type:ignore
import gensim.downloader as gensim_api #type:ignore

import numpy as np #type: ignore
import pandas as pd #type: ignore
import matplotlib.pyplot as plt #type:ignore
import re
from bs4 import BeautifulSoup as bs #type:ignore

def load_word2vec(model_name = 'word2vec-google-news-300'):
    model_path = gensim_api.load(model_name, return_path= True)
    word_vectors = KeyedVectors.load_word2vec_format(model_path, binary=
        ↪ True)
    return word_vectors

def clean_text(text):
    text = bs(text, 'html.parser').get_text()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text, flags=re.MULTILINE)
    text = re.sub(r"#\S+|\@\S+", "", text) # Removes hashtags & mentions
    text = re.sub(r"[^A-Za-z0-9\s]", "", text) # Keeps only alphanumeric
        ↪ characters and spaces
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()
    return text

def text_to_tokens(text, stopwords):
    get_stop_words = set(stopwords.words('english'))
    text = text.lower()
    tokens = re.findall(r'\w+', text)
    tokens = [token for token in tokens if token not in get_stop_words]
    return tokens

def split_data(data_path, train_size, test_size, seed):
    np.random.seed(seed)
    df = pd.read_csv(data_path)
    reviews, sentiments = df.review.values.tolist(),
        ↪ df.sentiment.values.tolist()
    # convert all sentiments to integers
    for i in range(len(sentiments)):
        sentiments[i] = 0 if sentiments[i] == 'negative' else 1

    random_indices = np.arange(len(reviews))
    np.random.shuffle(random_indices)
```



```

train_reviews, train_sentiments = [reviews[i] for i in
    ↪ random_indices[:train_size]], \
    [sentiments[i] for i in
    ↪ random_indices[:train_size]]
test_reviews, test_sentiments = [reviews[i] for i in
    ↪ random_indices[train_size:train_size + test_size]], \
    [sentiments[i] for i in
    ↪ random_indices[train_size:train_
    ↪ size+test_size]]
val_reviews, val_sentiments = [reviews[i] for i in
    ↪ random_indices[train_size + test_size:]], \
    [sentiments[i] for i in random_indic
    ↪ es[train_size+test_size:]]

sentiment_data = dict(
    train_reviews = train_reviews,
    train_sentiments = train_sentiments,
    test_reviews = test_reviews,
    test_sentiments = test_sentiments,
    val_reviews = val_reviews,
    val_sentiments = val_sentiments
)

return sentiment_data

def tokenize_data(document):
    '''
    document: should be a list of strings where each strings is one sample
    '''
    tokenized_document = []
    for sentence in document:
        tokenized = text_to_tokens(sentence)
        tokenized_document.append(tokenized)
    return tokenized_document

def get_info(x, y, data_type):
    print(f'\nDetails of {data_type} data: \n')
    assert len(x) == len(y)
    print(f'Number of Samples in {data_type}: {len(x)} ')
    print('Number of positive sentiments: ', np.sum([1 for i in y if i==1]))
    print('Number of negative sentiments: ', np.sum([1 for i in y if i==0]))

def clean_data(train, val, test):
    clean_xtrain = []
    for text in train:
        cleaned_text = clean_text(text)

```

```

        clean_xtrain.append(cleaned_text)

    clean_xval = []
    for text in val:
        cleaned_text = clean_text(text)
        clean_xval.append(cleaned_text)

    clean_xtest = []
    for text in test:
        cleaned_text = clean_text(text)
        clean_xtest.append(cleaned_text)

    return clean_xtrain, clean_xval, clean_xtest

def preprocess(datapath, train_size, test_size, print_info, seed):
    train_test_val_split = split_data(datapath, train_size, test_size, seed)

    ###---LOAD DATA IN DATA, LABEL FORMAT---###
    xtrain, ytrain = train_test_val_split['train_reviews'],
    ↪ train_test_val_split['train_sentiments']
    xtest, ytest = train_test_val_split['test_reviews'],
    ↪ train_test_val_split['test_sentiments']
    xval, yval = train_test_val_split['val_reviews'],
    ↪ train_test_val_split['val_sentiments']

    if print_info:
        get_info(xtrain, ytrain, 'Train')
        get_info(xtest, ytest, 'Test')
        get_info(xval, yval, 'Validation')

    xtrain, xval, xtest = clean_data(xtrain, xval, xtest)
    return [(xtrain, ytrain), (xval, yval), (xtest, ytest)]

if __name__ == '__main__':

    SEED = 746
    np.random.seed(SEED)

```

Dataset and Data-Loader Module

```

import numpy as np
import re
import os
from bs4 import BeautifulSoup as bs #type:ignore
import torch
import nltk #type:ignore

```

```

from torch.utils.data import Dataset, DataLoader

def create_vocabulary(text, cutoff_freq, word_vectors):
    word_list = []

    # Process each document in texts here and store the list of all words
    for x in text:
        filtered_tokens = nltk.word_tokenize(x, language= 'english')
        word_list += filtered_tokens

    unique_words, counts = np.unique(word_list, return_counts=True)
    vocab_dict = {word: count for word, count in zip(unique_words, counts) if
        ↪ (count >= cutoff_freq and word_vectors.__contains__(word))}

    # Update the key values to be indices rather than counts
    for counter, key in enumerate(vocab_dict):
        vocab_dict[key] = counter + 2

    vocab_dict = dict({'<PAD>':0, '<UNK>': 1}, **vocab_dict)

    return vocab_dict

def vocab_word2vec(vocab, word_vectors):
    # Get word2vec embedding matrices for words in vocab
    vocab_w2v = []
    dummy_linear = torch.nn.Linear(2, 300)
    for key in vocab:
        if key in ['<PAD>', '<UNK>']:
            vocab_w2v.append(dummy_linear.weight.data[:, vocab[key]])
        else:
            vocab_w2v.append(torch.tensor(word_vectors[key]))

    vocab_w2v = torch.stack(vocab_w2v, dim = 0)
    return vocab_w2v

# %%
class TokenizerDataset(Dataset):
    def __init__(self, text, labels, vocab, max_seq_len = None):
        self.text = text
        self.labels = labels
        self.vocab = vocab
        self.max_seq_len = max_seq_len
        nltk.download('stopwords')
        nltk.download('punkt_tab')
        from nltk.corpus import stopwords #type:ignore

    def encode(self, xbatch):

```

```

'''
xbatch: should be list of list of tokens
'''

encoded_xbatch = []
for entry in xbatch:
    tokenized_entry = nltk.word_tokenize(entry, language = 'english')
    encoded_xbatch.append([self.vocab.get(token, self.vocab['<UNK>'])
        ↪ for token in tokenized_entry])

return encoded_xbatch

def pad(self, encoded_xbatch):
    if self.max_seq_len is None:
        max_len = max([len(l) for l in encoded_xbatch])
    else:
        max_len = self.max_seq_len

    for i, entry in enumerate(encoded_xbatch):
        if len(entry) <= max_len:
            tokens = [*entry]
            tokens += ([self.vocab['<PAD>']]*(max_len - len(entry)))
        else:
            tokens = entry[:max_len]
        encoded_xbatch[i] = tokens
    return encoded_xbatch

def collate_fn(self, batch):
'''
data: should be list of cleaned strings
'''
max_length = 0

xbatch = [data[0] for data in batch]
ybatch = [data[1] for data in batch]

encoded_xbatch = self.encode(xbatch)
encoded_xbatch = self.pad(encoded_xbatch)
return torch.tensor(encoded_xbatch, dtype = torch.long),
    ↪ torch.tensor(ybatch)

def __len__(self):
    return len(self.text)

def __getitem__(self, idx):
    return self.text[idx], float(self.labels[idx])

def create_dataloader(dataset, batch_size, shuffle = True, collate_fn =
    ↪ None):

```

```
return DataLoader(dataset, batch_size= batch_size, shuffle = shuffle,
↪ collate_fn= collate_fn)
```

Trainer Module

```
import torch
from tqdm import tqdm
import os

class Accuracy:
    def accuracy_bce(preds, targets):
        preds = (preds > 0).to(torch.long)
        targets = targets.to(torch.long)
        acc = torch.sum(preds == targets)
        return acc.item()

    def accuracy_ce(preds, targets):
        preds = torch.argmax(preds, dim = 1)
        acc = torch.sum(preds == targets)
        return acc.item()

class Trainer:
    def __init__(self, model, criterion, optimizer, accuracy, save_dir,
↪ scheduler = None, use_tqdm = True):

        self.model          = model
        self.optimizer       = optimizer
        self.scheduler       = scheduler
        self.criterion      = criterion
        self.accuracy = accuracy
        self.loss_history    = { 'train': [], 'val': [] }
        self.acc_history    = { 'train': [], 'val': [] }
        self.device = model.device
        self.use_tqdm = use_tqdm
        self.save_dir = save_dir

    # def accuracy(self, preds, targets):
    #     #preds = torch.argmax(preds, dim = 1)
    #     #probs = torch.nn.Sigmoid()(preds)
    #     preds = (preds > 0).to(torch.long)
    #     targets = targets.to(torch.long)
    #     acc = torch.sum(preds == targets)
    #     return acc.item()

    def train_step(self, train_loader):
        self.model.train()
        train_loss = 0.0
        train_acc = 0
```

```

samples = 0
generator = tqdm(train_loader, desc = 'Processing Training Batch...')
↪ if self.use_tqdm else train_loader
for x, y in generator:
    x, y = x.to(self.device), y.to(self.device)
    preds = self.model(x)
    loss = self.criterion(preds,y)

    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    train_loss += loss.item()
    train_acc += self.accuracy(preds, y)
    samples += len(x)

if self.scheduler is not None:
    self.scheduler.step()
train_loss /= len(train_loader)
train_acc /= samples
return train_loss, train_acc

def eval_step(self, val_loader):
    self.model.eval()
    val_loss = 0.0
    val_acc = 0
    samples = 0
    generator = tqdm(val_loader, desc = 'Processing Val Batch...') if
↪ self.use_tqdm else val_loader
    with torch.no_grad():
        for x,y in generator:
            x, y = x.to(self.device), y.to(self.device)
            preds = self.model(x)
            val_loss += self.criterion(preds, y).item()
            val_acc += self.accuracy(preds, y)
            samples += len(x)

        val_loss /= len(val_loader)
        val_acc /= samples
    return val_loss, val_acc

def train(self, train_loader, val_loader, num_epochs):
    best_acc = 0
    model_path = os.path.join(self.save_dir, 'best.pth')
    for epoch in range(num_epochs):

        # train step
        train_loss, train_acc = self.train_step(train_loader)

```

```

self.loss_history['train'].append(train_loss)
self.acc_history['train'].append(train_acc)

# Val step
val_loss, val_acc = self.eval_step(val_loader)
self.loss_history['val'].append(val_loss)
self.acc_history['val'].append(val_acc)

if val_acc > best_acc:
    best_acc = val_acc
    torch.save(self.model.state_dict(), model_path)

print(f'Epoch: {epoch+1} | Train Loss: {train_loss:.4f} | Train
↪ Acc: {train_acc:.4f}\
    | Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}')

save_metric = dict(
    loss = self.loss_history,
    acc = self.acc_history,
    model_path = model_path,
)

return save_metric

```

Driver main code for running the modules and scripts for question-1

```

import prep_data #type:ignore
import dataset #type:ignore
import trainer #type:ignore
import models #type:ignore
import os

import numpy as np
import torch
import matplotlib.pyplot as plt

def get_modal_dict(model_name):
    model_dict = dict(
        lstm_without_attention = dict(
            vocab_size = len(vocab),
            embd_dims = 300,
            hidden_size = 256,
            word2vec_embd = vocab_w2v,
            num_layers = 2,
            dropout = 0.3

```

```

    ),
    lstm_with_attention = dict(
        vocab_size = len(vocab),
        embd_dims = 300,
        hidden_size = 256,
        word2vec_embd = vocab_w2v,
        num_layers = 2,
        dropout = 0.3
    ),
    transformer_encoder = dict(
        word2vec_embd = vocab_w2v,
        embed_dim = 300,
        hidden_dim = 256,
        mlp_size = 512,
        max_seq_len = 512,
        num_heads = 4,
        num_layers = 1,
        attn_dropout = 0.3,
        mlp_dropout = 0.3
    )
)
)
return model_dict[model_name]

def plot_losses(train_loss, val_loss, save_path, model_name):

    fig = plt.figure(figsize = (6,6))
    epochs = range(1, len(train_loss)+1)
    plt.plot(epochs, train_loss, color = 'blue', label = 'Train Loss')
    plt.plot(epochs, val_loss, color = 'green', label = 'Val Loss')
    plt.xlabel('Epochs')
    plt.xticks(epochs)
    plt.ylabel('Loss')
    plt.title(f'Loss vs Epoch plot for model: {model_name.replace("_", "
    ↵ ")})')
    plt.legend()
    save_path = os.path.join(save_path, 'loss_plot.png')
    plt.savefig(save_path)
    return

if __name__ == '__main__':

    print('All good')
    SEED = 746
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    device_idx = 6

```



```

device = torch.device(f'cuda:{device_idx}') if torch.cuda.is_available()
↳ else torch.device('cpu')
#device = torch.device('cpu')
print('Using device: ', device)

# prepare the data
datapath = '/data/home/samsadalam/MLSP_Assignment/Assignment3/IMDB
↳ Dataset.csv'
train_size = 40000
test_size = 5000
print_info = True

[(xtrain, ytrain), (xval, yval), (xtest, ytest)] =
↳ prep_data.preprocess(datapath, train_size, test_size, print_info,
↳ SEED)

# Get word2vec embeddings
print('\nLoading Word2Vec...')
word_vectors = prep_data.load_word2vec()

#Now create vocab and vocab embeddings from word2vec
print('\nCreating Vocab...')
if os.path.exists('/data/home/samsadalam/MLSP_Assignment/Assignment3/vocab_
↳ ab.pt'):
    vocab = torch.load('/data/home/samsadalam/MLSP_Assignment/Assignment_
↳ 3/vocab.pt', weights_only=
↳ False)
else:
    vocab = dataset.create_vocabulary(xtrain, cutoff_freq = 2,
↳ word_vectors = word_vectors)
    torch.save(vocab,
↳ '/data/home/samsadalam/MLSP_Assignment/Assignment3/vocab.pt' )

print('\nCreating Word2Vec for Vocab...')
if os.path.exists('/data/home/samsadalam/MLSP_Assignment/Assignment3/vocab_
↳ ab_w2v.pt'):
    vocab_w2v = torch.load('/data/home/samsadalam/MLSP_Assignment/Assign_
↳ ment3/vocab_w2v.pt', weights_only=
↳ False)
else:
    vocab_w2v = dataset.vocab_word2vec(vocab, word_vectors)
    torch.save(vocab_w2v,
↳ '/data/home/samsadalam/MLSP_Assignment/Assignment3/vocab_w2v.pt')

#prepare dataset and dataloader

train_dataset = dataset.TokenizerDataset(xtrain, ytrain, vocab)

```

```

val_dataset = dataset.TokenizerDataset(xval, yval, vocab)
test_dataset = dataset.TokenizerDataset(xtest, ytest, vocab)

train_loader = dataset.create_dataloader(train_dataset, batch_size = 32,
    ↪ shuffle = True, collate_fn= train_dataset.collate_fn)
val_loader = dataset.create_dataloader(val_dataset, batch_size= 32,
    ↪ shuffle = False, collate_fn= val_dataset.collate_fn)
test_loader = dataset.create_dataloader(test_dataset, batch_size= 32,
    ↪ shuffle = False, collate_fn= test_dataset.collate_fn)

model_name = 'transformer_encoder'
model_dict = get_modal_dict(model_name)

if model_name == 'lstm_without_attention':
    model = models.LSTMwithoutAttn(**model_dict).to(device)
elif model_name == 'lstm_with_attention':
    model = models.LSTMwithAttention(**model_dict).to(device)
elif model_name == 'transformer_encoder':
    model = models.TransformerModel(**model_dict).to(device)
    #print(model)
    max_seq_len = model_dict['max_seq_len']
    train_dataset.max_seq_len = max_seq_len
    val_dataset.max_seq_len = max_seq_len
    test_dataset.max_seq_len = max_seq_len
else:
    print('Model name error! Model not found.')
    exit()

optimizer = torch.optim.Adam(params = model.parameters(), lr = 0.00005,
    ↪ weight_decay= 0.00001)
criterion = torch.nn.BCEWithLogitsLoss()
accuracy = trainer.Accuracy.accuracy_bce

print('\nStarting the training...\n')
os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/'+
    ↪ model_name, exist_ok = True)
save_dir = '/data/home/samsadalam/MLSP_Assignment/Assignment3/'+
    ↪ model_name
model_trainer = trainer.Trainer(model, criterion, optimizer, accuracy,
    ↪ use_tqdm = False, save_dir = save_dir)
train_val_metric = model_trainer.train(train_loader, val_loader,
    ↪ num_epochs = 25)

# Compute test accuracy
best_model = train_val_metric['model_path']
model_state_dict = torch.load(best_model, map_location= device,
    ↪ weights_only= True)
model.load_state_dict(model_state_dict)

```

```

test_loss, test_acc = model_trainer.eval_step(test_loader)
print('\nTraining finished...\n')

# Plot train and val losses
train_loss = train_val_metric['loss']['train']
val_loss = train_val_metric['loss']['val']

plot_losses(train_loss, val_loss, save_dir, model_name.replace("_", " "))

with open(os.path.join(save_dir, 'test_acc.txt'), 'w') as f:
    f.write(f'Test Acc: {test_acc}')

```

LSTM Model with Average Pooling

```

import torch
import torch.nn as nn

class LSTMwithoutAttn(nn.Module):
    def __init__(self, vocab_size, embd_dims, hidden_size, word2vec_embd,
        ↪ num_layers = 2, dropout = 0.1):
        super(LSTMwithoutAttn, self).__init__()

        self._dummy = nn.Parameter(torch.Tensor(0), requires_grad=False)
        self.embedding = nn.Embedding.from_pretrained(word2vec_embd,
        ↪ freeze=False, padding_idx=0)
        self.lstm = nn.LSTM(embd_dims, hidden_size, batch_first=True,
        ↪ num_layers=num_layers, dropout=dropout)
        self.output = nn.Linear(hidden_size, 1)

    @property
    def device(self):
        return self._dummy.device

    def forward(self, x):
        x = self.embedding(x)
        out, hidden = self.lstm(x)
        # out = self.output(out[:, -1, :]).squeeze(dim = 1)
        out = self.output(torch.mean(out, dim=1)).squeeze(dim=1)
        return out

```

The plot of training and validation loss and the test accuracy are given below-

Test Accuracy of LSTM without Attention: 89.12%

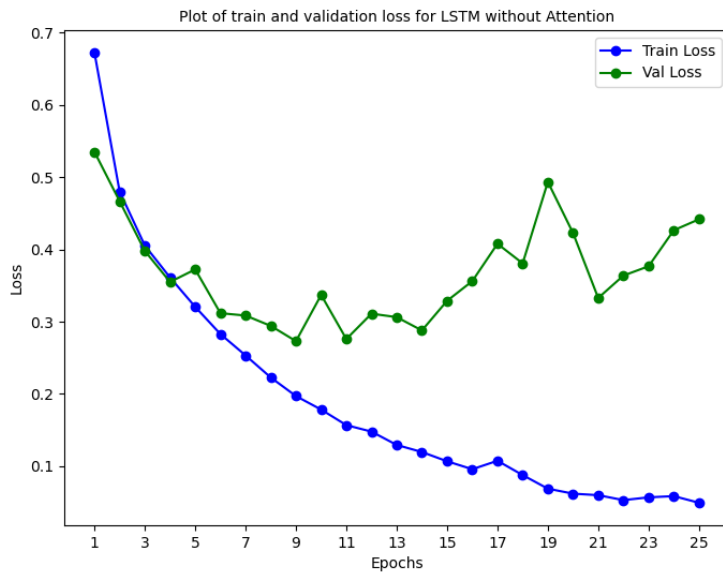


Figure 2: Plot of train and validation loss for LSTM with average pooling

(c) LSTM with Attention based Pooling

Model

```

class LSTMwithAttention(nn.Module):
    def __init__(self, vocab_size, embd_dims, hidden_size, word2vec_embd,
        ↪ num_layers = 2, dropout = 0.1):
        super(LSTMwithAttention, self).__init__()

        self._dummy = nn.Parameter(torch.Tensor(0), requires_grad=False)
        self.embedding = nn.Embedding.from_pretrained(word2vec_embd, freeze=
        ↪ True, padding_idx=0)
        self.lstm = nn.LSTM(embd_dims, hidden_size, batch_first= True,
        ↪ num_layers= num_layers, dropout= dropout)
        self.attention = nn.Linear(hidden_size, 1, bias = False)
        self.output = nn.Linear(hidden_size, 1)

    @property
    def device(self):
        return self._dummy.device

    def forward(self, x):
        x = self.embedding(x)
        out, hidden = self.lstm(x)
        attn_scores = self.attention(out)
        out = out.permute(0,2,1)
        weighted_out = torch.bmm(out, attn_scores).squeeze(dim = 2)
        out = self.output(weighted_out).squeeze(dim=1)

```

```
return out
```

The test accuracy and the plot of training and validation loss is shown in [Figure 3](#)

Test Accuracy of LSTM with Attention: 90.52%

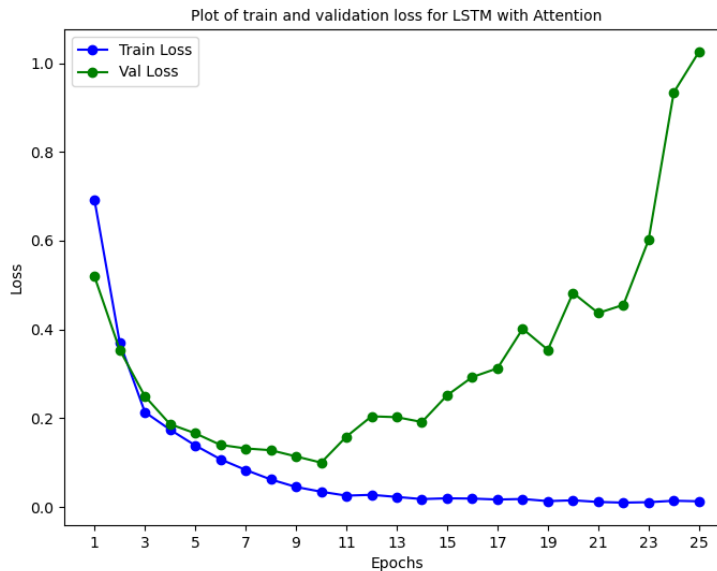


Figure 3: Plot of train and validation losses for LSTM with attention based pooling

Comparison of performance of the models - LSTM with Average Pooling and LSTM with Attention based Pooling

We can observe from the above outputs that the attention based average pooling is working better compared to the LSTM with average pooling. This is quite expected since attention based pooling decides which token are more important to decide the ultimate output, hence it learns to contextually weight the tokens instead of just averaging their outputs. This leads to better performance.

However, we see that the LSTM model with attention pooling is overfitting more than that of average pooling. This again can be attributed to attention based model learning quickly. We need to use slow learning rate or learning rate scheduler with regularization and early stopping to avoid this overfitting.

(d) Transformer Encoder

Model

```
# MULTIHEAD SELF ATTENTION BLOCK
```

```
class MSA(nn.Module):  
    def __init__(self, embed_dim, num_heads, attn_dropout):  
        super().__init__()
```

```

self.multi_head_self_attn = nn.MultiheadAttention(embed_dim =
↳ embed_dim, num_heads = num_heads, dropout = attn_dropout,
↳ batch_first= True)
self.layernorm = nn.LayerNorm(normalized_shape= embed_dim)

def forward(self, x):
    x = self.layernorm(x)
    x, _ = self.multi_head_self_attn(query = x, key = x, value = x,
↳ need_weights = False) # it returns both attn output and attn
↳ weights
    return x

# %%
# MULTI LAYER PERCEPTRON BLOCK
class MLP(nn.Module):
    def __init__(self, embed_dim, mlp_size, mlp_dropout):
        super().__init__()
        self.layernorm = nn.LayerNorm(normalized_shape = embed_dim)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, mlp_size),
            nn.ReLU(),
            nn.Dropout(p = mlp_dropout),
            nn.Linear(mlp_size, embed_dim),
            nn.Dropout(p = mlp_dropout)
        )

    def forward(self, x):
        x = self.layernorm(x)
        x = self.mlp(x)
        return x

# %%
# TRANSFORMER ENCODER USING MULTI-HEAD SELF ATTENTION
class TransformerEncoder(nn.Module):
    def __init__(self, embed_dim, mlp_size, num_heads, attn_dropout,
↳ mlp_dropout):
        super().__init__()

        self.msa_block = MSA(embed_dim, num_heads, attn_dropout)
        self.mlp_block = MLP(embed_dim, mlp_size, mlp_dropout)

    def forward(self, x):
        x = self.msa_block(x) + x
        x = self.mlp_block(x) + x
        return x

# %%
# COMPLETE TRANSFORMER MODEL

```

```

class TransformerModel(nn.Module):
    def __init__(self, word2vec_embd, embed_dim, hidden_dim, mlp_size,
        ↪ max_seq_len, num_heads, num_layers, attn_dropout, mlp_dropout):
        super(TransformerModel, self).__init__()

        self._dummy = nn.Parameter(torch.Tensor(0), requires_grad=False)
        self.position_embd = nn.Embedding(max_seq_len, embedding_dim=
            ↪ hidden_dim)
        self.max_seq_len = max_seq_len
        self.word_embd = nn.Embedding.from_pretrained(word2vec_embd,
            ↪ freeze=False, padding_idx=0)
        self.embd_projection = nn.Linear(embed_dim, hidden_dim)
        self.transformer_layer = nn.Sequential(*[
            TransformerEncoder(hidden_dim, mlp_size, num_heads, attn_dropout,
            ↪ mlp_dropout ) for _ in range(num_layers)
        ])
        self.output = nn.Linear(hidden_dim, 1)

    @property
    def device(self):
        return self._dummy.device

    def forward(self, x):
        x = self.word_embd(x)
        x = self.embd_projection(x)
        x = self.position_embd(torch.arange(self.max_seq_len).to(self.device)
            ↪ )) +
            ↪ x
        x = self.transformer_layer(x)
        x = self.output(torch.mean(x, dim = 1)).squeeze(dim=1)
        return x

```

The training accuracy and plot of the train and validation loss are shown in the [Figure 4](#)

Test Accuracy of Transformer Encoder Model: 89.02%

Observations on outputs of all above models:

We can see that the LSTM model with attention based pooling works best. Though we would expect the transformer to work best. This conflicting observation may be because of the way the transformer model is trained. The performance of a transformer model is based on how we train the model and the size of the dataset. If we can maintain training stability without overfitting for more number of epochs, we would be able to achieve better accuracy with transformer model.

2 Time Delay Neural Network

Notations:

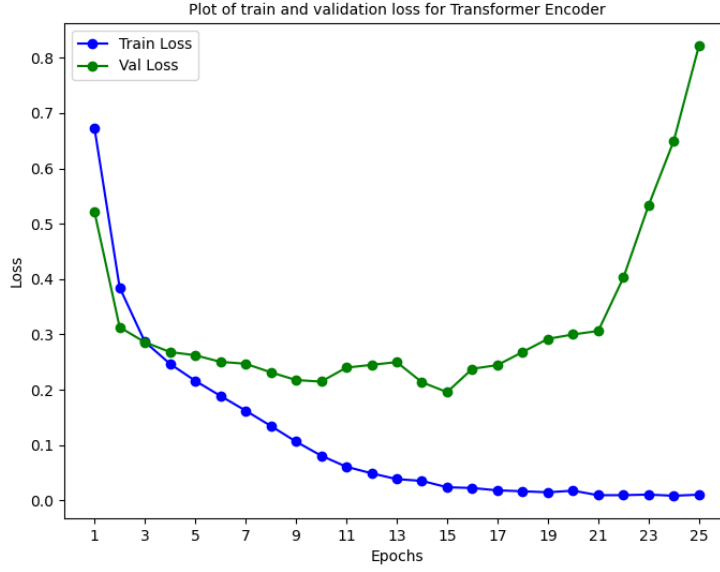


Figure 4: Plot of train loss and validation loss for Transformer Encoder model

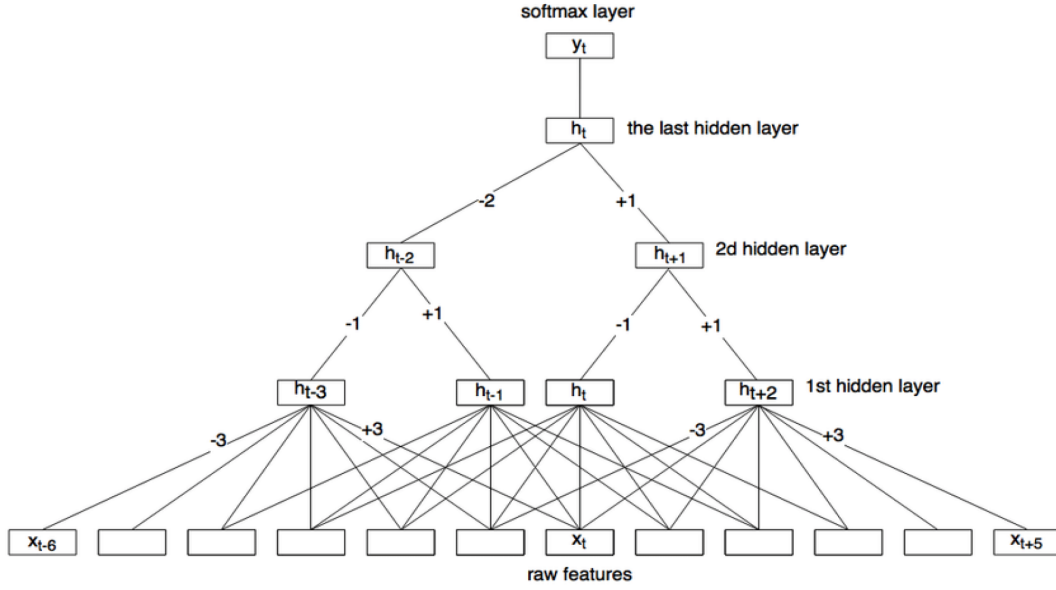


Figure 5: Time Delay Neural Network

Assume that $\mathbf{x}_t \in \mathbb{R}^D$, hidden outputs in layer l are $\mathbf{h}_t \in \mathbb{R}^l$, $\mathbf{y}_t \in \mathbb{R}^C$ where C is the number of output classes. Let H_l denote the dimension of hidden state vectors in l th layer. Let $\mathbf{a}_t^l$ denote the pre-activation output before applying non-linearity.

Number of Parameters:

let C_l be the context length from $l - 1$ th layer to l th layer. Each neuron in the $(l - 1)$ th layer is connected to each neuron in l th layer provided there is a solid connection from the neuron in $(l - 1)$ th layer to the said neuron in l th layer. Considering the inputs and hidden states to be vectors and the connection between them to be dense then for every solid connection from $(l - 1)$ th layer

to l th is parameterized by a weights matrix of dimension $H_l \times H_{l-1}$. Then the number of weight parameters from $l - 1$ th layer to l th layer will be - $H_l \times H_{l-1} \times C_l$. Using this logic, the number of weight parameters in different layers of the network shown is -

1. Weight parameters for connection from input layer to first hidden layer = $7DH_1$
2. Weight parameters for connection from first hidden layer to second hidden layer = $2H_1H_2$
3. Weight parameters for connection from second hidden layer to third hidden layer = $2H_2H_3 = 2H_2C$ since $H_3 \in \mathbb{R}^C$
4. No parameters in softmax layer.

$$\text{Total number of parameters} = 7DH_1 + 2H_1H_2 + 2H_2C$$

If we also assume bias parameters associated with each linear layer, then total number of parameters become -

$$\text{Total number of parameters} = 7(DH_1 + H_1) + 2(H_1H_2 + H_2) + 2(H_2C + C)$$

Backpropagation through TDNN

The output of any hidden layer neuron can be written as -

$$\begin{aligned} \mathbf{a}_t^{(l)} &= \sum_{\tau \in C(l)} \mathbf{W}_\tau^{(l)} \mathbf{h}_{t+\tau}^{(l-1)} \\ \mathbf{h}_t^{(l)} &= \mathbf{g}(\mathbf{a}_t^{(l)}) \end{aligned}$$

where $\mathbf{g}(\cdot)$ is activation function (ReLU) here.

$$\mathbf{h}_t^{(l)} = \mathbf{g} \left(\sum_{\tau \in C(l)} \mathbf{W}_\tau^{(l)} \mathbf{h}_{t+\tau}^{(l-1)} \right) \quad (1)$$

where the set $C(l)$ is the set to time indices which fall under context of input for l th layer. For example $C(1) = \{-3, -2, \dots, 2, 3\}$, $C(2) = \{-1, 1\}$, $C(3) = \{-2, 1\}$ for TDNN in Figure 5. Clearly, $\mathbf{h}_t^{(0)} = \mathbf{x}_t$ and $l = 0, 1, 2, 3$ in our case. $\mathbf{g}(\cdot)$ is activation function. The shape of the different parameters used in the above equation is -

$$\mathbf{W}_\tau^l \in \mathbb{R}^{H_l \times H_{(l-1)}}, \mathbf{a}_t^l \in \mathbb{R}^l, \mathbf{h}_t^l \in \mathbb{R}^{h_l}$$

We continue this forward computation till the last hidden layer and then apply softmax non-linearity -

$$\begin{aligned} \mathbf{y}_t &= \text{softmax}(\mathbf{h}_t) \\ \Rightarrow y_{t,i} &= \frac{\exp(h_{t,i})}{\sum_{j=1}^C \exp(h_{t,j})} \quad \forall i = 1, 2, \dots, C \end{aligned}$$

and the compute cross-entropy loss as -

$$CE(\mathbf{y}_t, \mathbf{z}_t) = - \sum_{k=1}^C z_{t,k} \log(y_{t,k})$$

For example, for out network, we can write the forward pass equations as -

For the outputs $\mathbf{h}_t$ in the first layer -

$$\mathbf{h}_\tau = \text{ReLU}(\mathbf{W}_{-3}^1 \mathbf{x}_{\tau-3} + \mathbf{W}_{-2}^1 \mathbf{x}_{\tau-2} + \mathbf{W}_{-1}^1 \mathbf{x}_{\tau-1} + \mathbf{W}_0^1 \mathbf{x}_\tau + \mathbf{W}_1^1 \mathbf{x}_{\tau+1} + \mathbf{W}_2^1 \mathbf{x}_{\tau+2} + \mathbf{W}_3^1 \mathbf{x}_{\tau+3})$$

$$\forall \tau = t-3, t-1, t, t+2$$

For the output $\mathbf{h}_{t-2}$ and $\mathbf{h}_{t+1}$ in the second hidden layer -

$$\mathbf{h}_{t-2} = \text{ReLU}(\mathbf{W}_{-1}^2 \mathbf{h}_{t-3} + \mathbf{W}_1^2 \mathbf{h}_{t-1})$$

$$\mathbf{h}_{t+1} = \text{ReLU}(\mathbf{W}_{-1}^2 \mathbf{h}_t + \mathbf{W}_1^2 \mathbf{h}_{t+2})$$

For the outputs $\mathbf{h}_t$ in the third hidden layer -

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_{-2}^3 \mathbf{h}_{t-2} + \mathbf{W}_{+1} \mathbf{h}_{t+1})$$

and then we compute softmax and cross entropy using their respective equations as derived above.

This I did for illustration with the given network. However, I will stick to my original notation as described in the **Notations** part and general feedforward [Equation 1](#). I will also use preactivation output $\mathbf{a}_t$ and activation function $\mathbf{g}$ in general to simplify the mathematical treatment.

Backpropagation through layers

The error is given as -

$$E = \sum_{t=1}^T E_t = \sum_{t=1}^T CE(\mathbf{y}_t, \mathbf{z}_t)$$

Backpropagation through Error layer:

Here we are interested to compute $\frac{\partial E}{\partial \mathbf{h}_t}$, however, only E_t depends upon $\mathbf{h}_t$, this is same as computing $\frac{\partial E_t}{\partial \mathbf{h}_t}$. This is a vector whose i th component can be computed as -

$$\begin{aligned} \frac{\partial E_t}{\partial y_{t,i}} &= \frac{\partial CE(\mathbf{y}_t, \mathbf{z}_t)}{\partial y_{t,i}} \\ &= - \frac{\partial}{\partial y_{t,i}} \sum_{k=1}^C z_{t,k} \log(y_{t,k}) \\ &= - \frac{\delta_{i,c}}{y_{t,i}} \end{aligned}$$

where $\delta_{i,c}$ is **dirac-delta** function. So the gradient vector $\left[\frac{\partial E_t}{\partial \mathbf{h}_t} \right]$ is a vector of all zeros except for one non-zero entry given by $-\frac{1}{y_{t,c}}$ where c is the correct class of $\mathbf{z}_t$.

Backpropagation through softmax

Let the gradient of output with respect to the hidden state of layer-3 is $\frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t}$ which will be a matrix of the order $\mathbb{R}^{C \times C}$. The ij th entry of this matrix is -

$$\frac{\partial y_{t,i}}{\partial h_{t,j}^L} = y_{t,i}(\delta_{ij} - y_{t,j})$$

$$\frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t^L} = \text{diag}(\mathbf{y}_t) - \mathbf{y}_t \mathbf{y}_t^\top$$

So the gradient of the error with respect to the final hidden state is -

$$\begin{aligned} \frac{\partial E_t}{\partial \mathbf{h}_t^L} &= \frac{\partial E_t}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t^L} \\ &= -[\text{diag}(\mathbf{y}_t) - \mathbf{y}_t \mathbf{y}_t^\top] \left[\frac{\delta_{t,c}}{\mathbf{y}_t} \right] \\ &= \mathbf{y}_t - \mathbf{z}_t \end{aligned}$$

The gradient of total error $E = \sum_{t=1}^T E_t$ with respect to the hidden state $\mathbf{h}_t^L$ will be same as -

$$\frac{\partial E}{\partial \mathbf{h}_t^L} = \frac{\partial E_t}{\partial \mathbf{h}_t^L} = \mathbf{y}_t - \mathbf{z}_t \quad (2)$$

Now we proceed to compute the gradients with respect to hidden states in deeper layers and with respect to weights in those layers.

Backpropagation with respect to weights of the layer

Here we are interested to find the gradients $\left[\frac{\partial E_t}{\partial \mathbf{W}_\tau^1} \right]$ assuming that we know the gradients $\left[\frac{\partial E_t}{\partial \mathbf{h}_t^l} \right]$. This can be computed using chain rule of the gradient. Note that the weights affect hidden states corresponding to all time steps because of weights sharing. The backpropagation will also trace back to $\mathbf{W}_\tau^1$ through all hidden states $\mathbf{h}_t$ $\forall t = 1, 2, \dots, T$. This can be written as -

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{W}_\tau^1} &= \sum_{t=1}^T \frac{\partial E_t}{\partial \mathbf{h}_t^1} \frac{\partial \mathbf{h}_t^1}{\partial \mathbf{W}_\tau^1} \\ &= \sum_{t=1}^T \frac{\partial E_t}{\partial \mathbf{h}_t^1} \frac{\partial \mathbf{h}_t^1}{\partial \mathbf{a}_t^1} \frac{\partial \mathbf{a}_t^1}{\partial \mathbf{W}_\tau^1} \\ &= \sum_{t=1}^T \left[\frac{\partial E_t}{\partial \mathbf{h}_t^1} \circ (\mathbf{g}^1)' \right] \frac{\partial \mathbf{a}_t^1}{\partial \mathbf{W}_\tau^1} \end{aligned}$$

where ' $\circ$ ' denotes element-wise multiplication between two vectors of same dimension. $(\mathbf{g}^l)'$ denotes the gradient of the activation function with respect to pre-activation output $\mathbf{a}_t^l$ in l th layer. For the sake of completeness of proof, here I give this gradient also -

$$(\mathbf{g}^l)' = \frac{\partial \mathbf{g}^l}{\partial \mathbf{a}_t^l} = \begin{cases} 1 & \text{if } \mathbf{a}_t^l > 0 \\ 0 & \text{if } \mathbf{a}_t^l \leq 0 \end{cases}$$

Note that the comparison operators are used with vectors and hence are meant for element-wise comparison.

Using the above results we can finally write the gradient with respect to weights -

$$\frac{\partial E}{\partial \mathbf{W}_\tau^1} = \sum_{t=1}^T \left[\frac{\partial E_t}{\partial \mathbf{h}_t^1} \circ (\mathbf{g}^1)' \right] \cdot (\mathbf{h}_{t+\tau}^{1-1})^\top \quad (3)$$

backpropagation with respect to the output of the previous layer

Now we have gradient with respect to l th layer hidden states and weights, we need to compute gradient with respect to $(l-1)$ th layer hidden state so that we can backpropagate for all values of l . Hence, we are interested to compute the partial derivative $\left[\frac{\partial E_t}{\partial \mathbf{h}_t^{l-1}} \right]$. This can be computed as follows -

$$\frac{\partial E_t}{\partial \mathbf{h}_t^{l-1}} = \sum_{\tau \in C(l)} \frac{\partial E_t}{\partial \mathbf{h}_\tau^l} \frac{\partial \mathbf{h}_\tau^l}{\partial \mathbf{h}_t^{l-1}}$$

Where the summation is for all $\tau \in C(l)$ because the hidden state $\mathbf{h}_t^{l-1}$ affects all hidden states $\mathbf{h}_\tau^l \quad \forall \quad \tau \in C(l)$ as per the [Equation 1](#). So the summation is natural consequence of how we compute total derivative. Proceeding with this and using the [Equation 1](#)

$$\frac{\partial \mathbf{h}_\tau^l}{\partial \mathbf{h}_t^{l-1}} = \mathbf{W}_{t-\tau}^\top$$

where the subscript has been interchanged making τ appear with $\mathbf{h}_t^l$. This leads to subscript $(t-\tau)$ with $\mathbf{W}$ to satisfy [Equation 1](#).

Finally we get -

$$\frac{\partial E_t}{\partial \mathbf{h}_t^{l-1}} = \sum_{\tau \in C(l)} [\mathbf{W}_{t-\tau}^\top] \frac{\partial E_t}{\partial \mathbf{h}_\tau^l} \quad (4)$$

This completes the backpropagation through one layer. Now we can use the [Equation 3](#) and [Equation 4](#) repeatedly to backpropagate through deeper layers in the network. □

3 Audio Classification with CNNs

For this part, the trainer code remains same as the [Question-1](#). We create separate dataset for loading audio datasets and data-loaders.

Scripts and Modules

Datasets and Data-Loaders

```
# AUDIO DATASET
class AudioDataset(Dataset):
    def __init__(self, audiopath, df, target_to_idx, load_audio, win_shift):
```

```

        self.audiopath = audiopath
        self.df = df
        self.target_to_idx = target_to_idx
        self.load_audio = load_audio
        self.win_shift = win_shift

    def __len__(self):
        return self.df.shape[0]

    def __getitem__(self, index):
        audio_file = self.df['filename'].iloc[index]
        path_to_file = os.path.join(self.audiopath, audio_file)
        x = self.load_audio(path_to_file, self.win_shift)
        x = np.expand_dims(x, axis = 0)
        y = self.df['target'].iloc[index]
        y = self.target_to_idx[y]
        return x, y

# %%
def create_dataloader(dataset, batch_size, shuffle = True, collate_fn =
↳ None):
    return DataLoader(dataset, batch_size= batch_size, shuffle = shuffle,
↳ collate_fn= collate_fn)

```

CNN Model for all parts of question-3

```

# CNN Model for Q-3
class AudioCNN(nn.Module):
    def __init__(self, filter_depth, hidden_dims, num_classes, norm_type =
↳ None):
        super(AudioCNN, self).__init__()

        self._dummy = nn.Parameter(torch.Tensor(0), requires_grad=False)

        self.cnn = nn.Sequential(
            nn.Conv2d(1, filter_depth, 3, 1),
            self.norm1,
            nn.ReLU(),
            nn.MaxPool2d(3,3),
            nn.Conv2d(filter_depth, 16, 3, 1),
            self.norm2,
            nn.ReLU(),
            nn.MaxPool2d(3, 3)
        )
        self.flatten = nn.Flatten(start_dim= 1, end_dim= 3)

        if (norm_type is not None and norm_type == 'layernorm'):
            self.norm = nn.LayerNorm(normalized_shape= filter_depth*13*54)

```

```

        elif norm_type is not None and norm_type == 'batchnorm':
            self.norm = nn.BatchNorm1d(filter_depth*13*54)
        else:
            self.norm = nn.Identity()

    self.fcn = nn.Sequential(
        nn.Linear(filter_depth*13*54, hidden_dims),
        nn.ReLU(),
        nn.Linear(hidden_dims, num_classes),
    )

    @property
    def device(self):
        return self._dummy.device

    def forward(self, x):
        x = self.cnn(x)
        x = self.flatten(x)
        x = self.norm(x)
        x = self.fcn(x)
        return x

```

Driver main code for question-3

:

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd #type: ignore
import librosa #type: ignore
import os
import dataset
import models
import trainer
import torch

# %%
def get_device(device_idx):
    return torch.device(f'cuda:{device_idx}') if torch.cuda.is_available()
    ↪ else torch.device('cpu')

def load_df(dfpath):
    df = pd.read_csv(dfpath)
    esc10_df = df[df['esc10']]
    train_df = esc10_df[(esc10_df['fold']==1) | (esc10_df['fold']==2) |
    ↪ (esc10_df['fold']==3) ]
    val_df = esc10_df[(esc10_df['fold']==4)]

```

```

test_df = esc10_df[(esc10_df['fold']==5)]
return train_df, val_df, test_df

# %%
# LOAD AUDIO
def load_audio(filepath, win_shift = 10):
    '''
    win_shift: window shift in ms
    '''
    audio, sr = librosa.load(filepath, sr = None)
    n_fft = sr * win_shift // 1000
    hop_length = sr * win_shift // 1000
    mel_spec = librosa.feature.melspectrogram(y = audio, sr = sr, n_fft =
    ↪ n_fft, hop_length = hop_length)
    mel_spec = librosa.power_to_db(mel_spec, ref=np.max)
    return mel_spec

def get_optimizer(choice, lr, momentum, model):

    if choice == 'sgd':
        optimizer = torch.optim.SGD(model.parameters(), lr = lr)
    elif choice == 'sgd_with_momentum':
        optimizer = torch.optim.SGD(model.parameters(), lr = lr, momentum=
        ↪ momentum)
    elif choice == 'rms_prop':
        optimizer = torch.optim.RMSprop(model.parameters(), lr = lr)
    else:
        #print('Invalid Choice of Optimizer! Will return Adam')
        optimizer = torch.optim.Adam(model.parameters(), lr = lr)
    return optimizer

def plot_losses(train_loss, val_loss, save_path, plot_title):

    fig = plt.figure(figsize = (10,6))
    epochs = range(1, len(train_loss)+1)
    plt.plot(epochs, train_loss, color = 'blue', label = 'Train Loss')
    plt.plot(epochs, val_loss, color = 'green', label = 'Val Loss')
    plt.xlabel('Epochs')
    plt.xticks(epochs)
    plt.ylabel('Loss')
    plt.title(f'Loss vs Epoch plot for {plot_title}')
    plt.legend()
    save_path = os.path.join(save_path, 'loss_plot.png')
    plt.savefig(save_path)
    return

if __name__ == '__main__':

```

```

# %%      # DEFINE HYPERPARAMS
WINDOW_SHIFT = 10
BATCH_SIZE = 16
FILTER_DEPTH = 16
NUM_CLASSES = 10
HIDDEN_DIMS = 128
LR = 0.0005
MOMENTUM = 0.9
OPTIM_CHOICE = 'rms_prop'
NORM_CHOICE = 'no_norm'
NUM_EPOCHS = 20
SEED = 746

np.random.seed(SEED)
torch.manual_seed(SEED)

# Get the device
device = get_device(4)

# %%      # Load the DFs
print('Loading the data...')
df_path = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-mast_
↳ er/meta/esc50.csv'
train_df, val_df, test_df = load_df(df_path)

# Get the target indices, we need to convert the indices to 0-num_classes
↳ range
train_targets = train_df['target']
targets_to_index = {target: index for target, index in
↳ zip(train_targets.unique(), range(len(train_targets.unique())))}
#print(targets_to_index)

# Now prepare dataset and dataloader
print('Create dataset and dataloaders...')
audiopath = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-ma_
↳ ster/audio'

train_dataset = dataset.AudioDataset(audiopath, train_df,
↳ targets_to_index, load_audio, win_shift= WINDOW_SHIFT)
val_dataset = dataset.AudioDataset(audiopath, val_df, targets_to_index,
↳ load_audio, win_shift= WINDOW_SHIFT)
test_dataset = dataset.AudioDataset(audiopath, test_df, targets_to_index,
↳ load_audio, win_shift= WINDOW_SHIFT)

train_loader = dataset.create_dataloader(train_dataset, batch_size=
↳ BATCH_SIZE, shuffle = True, collate_fn= None)

```



```

val_loader = dataset.create_dataloader(val_dataset, batch_size=
    ↪ BATCH_SIZE, shuffle = False, collate_fn= None)
test_loader = dataset.create_dataloader(test_dataset, batch_size=
    ↪ BATCH_SIZE, shuffle = False, collate_fn= None)

# %% BASIC MODEL TRAINING FOR A AND B PART
model = models.AudioCNN(FILTER_DEPTH, HIDDEN_DIMS, NUM_CLASSES,
    ↪ NORM_CHOICE).to(device)
optimizer = get_optimizer(OPTIM_CHOICE, LR, MOMENTUM, model)
#scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size= 5,
    ↪ gamma = 0.2)
scheduler = None
criterion = torch.nn.CrossEntropyLoss()
accuracy = trainer.Accuracy.accuracy_ce

os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-ma
    ↪ ster/q3_cnn_b' + NORM_CHOICE+ '_' +OPTIM_CHOICE, exist_ok=
    ↪ True)
save_dir = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-mas
    ↪ ter/q3_cnn_b' + NORM_CHOICE+ '_' +
    ↪ OPTIM_CHOICE

# Begin training
print('Begin Training...')
model_trainer = trainer.Trainer(model = model,
                                criterion = criterion,
                                optimizer = optimizer,
                                accuracy = accuracy,
                                save_dir= save_dir,
                                scheduler = scheduler,
                                use_tqdm = True)

train_val_metric = model_trainer.train(train_loader, val_loader,
    ↪ num_epochs = NUM_EPOCHS)

# Compute test accuracy
best_model = train_val_metric['model_path']
model_state_dict = torch.load(best_model, map_location= device,
    ↪ weights_only= True)
model.load_state_dict(model_state_dict)

test_loss, test_acc = model_trainer.eval_step(test_loader)
print('\nTraining finished...\n')

# Plot train and val losses
train_loss = train_val_metric['loss']['train']
val_loss = train_val_metric['loss']['val']

```

```

train_acc = train_val_metric['acc']['train']
val_acc = train_val_metric['acc']['val']

plot_losses(train_loss, val_loss, save_dir, f'Adam and {NORM_CHOICE} ')

with open(os.path.join(save_dir, 'loss_acc.txt'), 'w') as f:
    for i in range(len(train_loss)):
        f.write(f'Epoch: {i+1} | Train Loss: {train_loss[i]:.4f} | Val
        ↳ Loss: {val_loss[i]:.4f} \n')
        f.write(f'Test Acc: {test_acc}')

print('Test Accuracy: ', test_acc)

```

For training the ensemble model following script is used -

```

# %% Train 3 Models and use them for ensembling

model_sgd_wo_norm = models.AudioCNN(FILTER_DEPTH, HIDDEN_DIMS,
↳ NUM_CLASSES, None).to(device)
optimizer_sgd_wo_norm = get_optimizer('sgd', 0.01, MOMENTUM,
↳ model_sgd_wo_norm)
os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-master/q3_cnn_' + 'c_sgd_wo_norm', exist_ok=
↳ True)
save_dir_sgd_wo_norm = '/data/home/samsadalam/MLSP_Assignment/Assignment
↳ 3/ESC-50-master/q3_cnn_' +
↳ 'c_sgd_wo_norm'

# Begin training
print('Begin Training SGD without Norm...')
model_trainer = trainer.Trainer(model = model_sgd_wo_norm,
                                criterion = criterion,
                                optimizer = optimizer_sgd_wo_norm,
                                accuracy = accuracy,
                                save_dir= save_dir_sgd_wo_norm,
                                scheduler = scheduler,
                                use_tqdm = True)

train_val_metric = model_trainer.train(train_loader, val_loader,
↳ num_epochs = NUM_EPOCHS)

# Compute test accuracy
best_model_sgd_wo_norm = train_val_metric['model_path']

# %% CNN with RMS Prop and Layernorm

```

```

model_rms_layern = models.AudioCNN(FILTER_DEPTH, HIDDEN_DIMS,
    ↪ NUM_CLASSES, norm_type = 'layernorm').to(device)
optimizer_rms_layern = get_optimizer('rms_prop', 0.0005, MOMENTUM,
    ↪ model_rms_layern)
os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-master/q3_cnn_' + 'c_rms_layernorm', exist_ok=
    ↪ True)
save_dir_rms_layern = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-master/q3_cnn_' +
    ↪ 'c_rms_layernorm'

# Begin training
print('Begin Training RMS Prop with Layernorm...')
model_trainer = trainer.Trainer(model = model_rms_layern,
                                criterion = criterion,
                                optimizer = optimizer_rms_layern,
                                accuracy = accuracy,
                                save_dir= save_dir_rms_layern,
                                scheduler = scheduler,
                                use_tqdm = True)

train_val_metric = model_trainer.train(train_loader, val_loader,
    ↪ num_epochs = NUM_EPOCHS)

# Compute test accuracy
best_model_rms_layern = train_val_metric['model_path']

# %%
# Adam with Batchnorm
# CNN with RMS Prop and Layernorm
model_adam_bn = models.AudioCNN(FILTER_DEPTH, HIDDEN_DIMS, NUM_CLASSES,
    ↪ norm_type = 'batchnorm').to(device)
optimizer_adam_bn = get_optimizer('adam', 0.0005, MOMENTUM,
    ↪ model_adam_bn)
os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-master/q3_cnn_' + 'c_adam_bn', exist_ok=
    ↪ True)
save_dir_adam_bn = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-master/q3_cnn_' +
    ↪ 'c_adam_bn'

# Begin training
print('Begin Training RMS Prop with Layernorm...')
model_trainer = trainer.Trainer(model = model_adam_bn,
                                criterion = criterion,
                                optimizer = optimizer_adam_bn,
                                accuracy = accuracy,

```

```

        save_dir= save_dir_adam_bn,
        scheduler = scheduler,
        use_tqdm = True)

train_val_metric = model_trainer.train(train_loader, val_loader,
    ↪ num_epochs = NUM_EPOCHS)

# Compute test accuracy
best_model_adam_bn = train_val_metric['model_path']

# %%
# Once the models are trained take their best state dict and load them
model_sgd_wo_norm.load_state_dict(torch.load(best_model_sgd_wo_norm,
    ↪ weights_only= True, map_location= device))
model_rms_layern.load_state_dict(torch.load(best_model_rms_layern,
    ↪ weights_only= True, map_location= device))
model_adam_bn.load_state_dict(torch.load(best_model_adam_bn,
    ↪ weights_only= True, map_location= device))
os.makedirs('/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-mas',
    ↪ ster/q3_cnn_ensemble_non_train', exist_ok=
    ↪ True)
save_dir = '/data/home/samsadalam/MLSP_Assignment/Assignment3/ESC-50-mas',
    ↪ ter/q3_cnn_ensemble_non_train'
# Create ensemble model
ensemble_model = models.EnsembleModel(model_sgd_wo_norm,
    ↪ model_rms_layern, model_adam_bn).to(device)
ensemble_trainer = trainer.Trainer(model = ensemble_model,
    criterion= criterion,
    optimizer= torch.optim.SGD(ensemble_model
    ↪ 1.parameters(), lr = 0.01, momentum=
    ↪ MOMENTUM),
    accuracy= accuracy,
    save_dir = save_dir,
    scheduler= None,
    use_tqdm = False
)

# Since we don't need to train, simply evaluate the ensemble model on the
    ↪ test set to get average accuracy
test_loss, test_acc_no_train = ensemble_trainer.eval_step(test_loader)
# with open(os.path.join(save_dir, 'test_acc.txt'), 'w') as f:
#     f.write(f'Test Acc: {test_acc}')

# print('Test Accuracy of Ensemble Averaging: ', test_acc)

# Now train the ensemble model for values of alpha but use validation set
    ↪ for training

```

```

train_val_metric = ensemble_trainer.train(val_loader, test_loader,
    ↪ num_epochs= 20)
ensemble_best = train_val_metric['model_path']
ensemble_model.load_state_dict(torch.load(ensemble_best, weights_only=
    ↪ True, map_location= device))

test_loss, test_acc_train = ensemble_trainer.eval_step(test_loader)
with open(os.path.join(save_dir, 'test_acc.txt'), 'w') as f:
    f.write(f'Test Acc without ensemble train: {test_acc_no_train}
    ↪ \nTest Acc with ensemble train: {test_acc_train}')

print(f'Test Acc without ensemble train: {test_acc_no_train} \nTest Acc
    ↪ with ensemble train: {test_acc_train}')

```

Results and Plots

(a) Plot of train and validation loss for CNN model for different optimizers and test accuracy

Test Accuracy for CNN with SGD without momentum: 53.75%
 Test Accuracy for CNN with SGD with momentum: 58.75%
 Test Accuracy for CNN with RMSprop: 66.25%
 Test Accuracy for CNN with Adam: 56.25%

(b) Plot of Train and Validation with choice of norm applied

The test accuracy obtained and the required plots are shown in -

Test Accuracy with no-norm: 56.25%
 Test Accuracy with layernorm: 62.50%
 Test Accuracy with batchnorm: 58.75%

(c) Model Ensembling

(c)-1 Output Averaging of the posterior model outputs

Ensemble Model code

```

# %%
# ENSEMBLE MODEL TRAINING
class EnsembleModel(nn.Module):
    def __init__(self, model1, model2, model3):
        super(EnsembleModel, self).__init__()
        self.model1 = model1
        self.model2 = model2

```

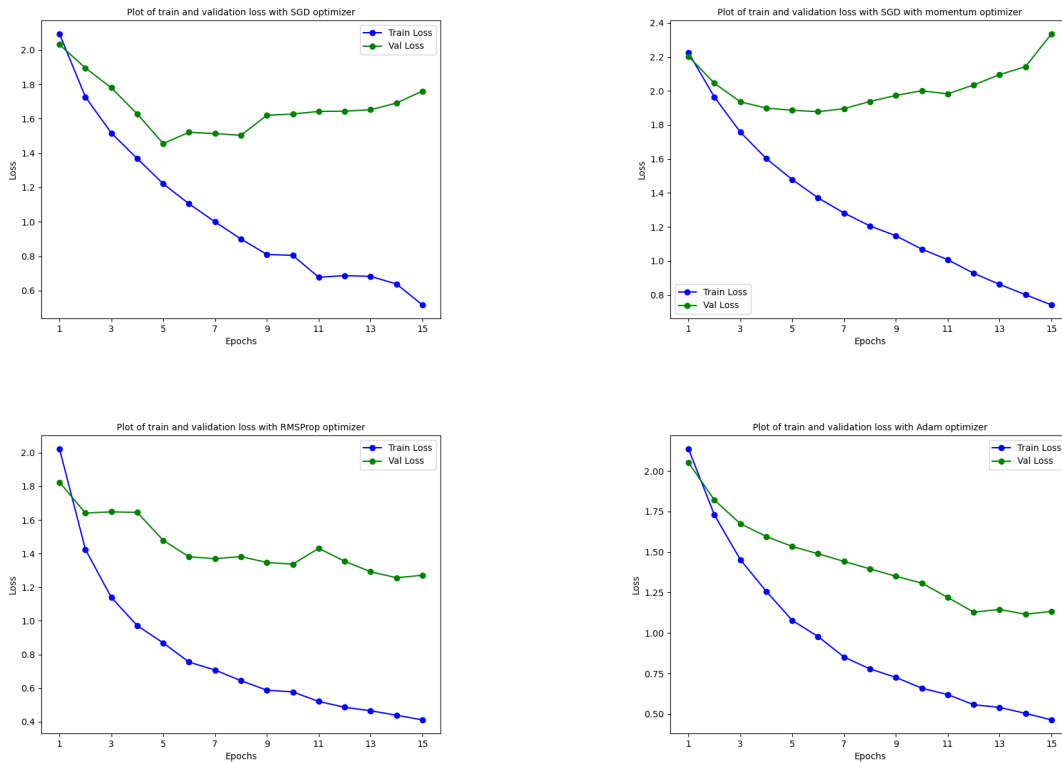


Figure 6: Plot of Training and Validation loss achieved with different optimizers

```

self.model3 = model3
self._dummy = nn.Parameter(torch.Tensor(0), requires_grad=False)

for param in model1.parameters():
    param.requires_grad = False
for param in model2.parameters():
    param.requires_grad = False
for param in model3.parameters():
    param.requires_grad = False

self.weights = nn.Parameter(data = torch.tensor([1/3, 1/3, 1/3]),
    ↪ requires_grad = True)

@property
def device(self):
    return self._dummy.device

def forward(self, x):
    y1 = self.model1(x)
    y2 = self.model2(x)
    y3 = self.model3(x)
    alpha = torch.nn.functional.softmax(self.weights)
    return (alpha[0]*y1 + alpha[1]*y2 + alpha[2]*y3)

```

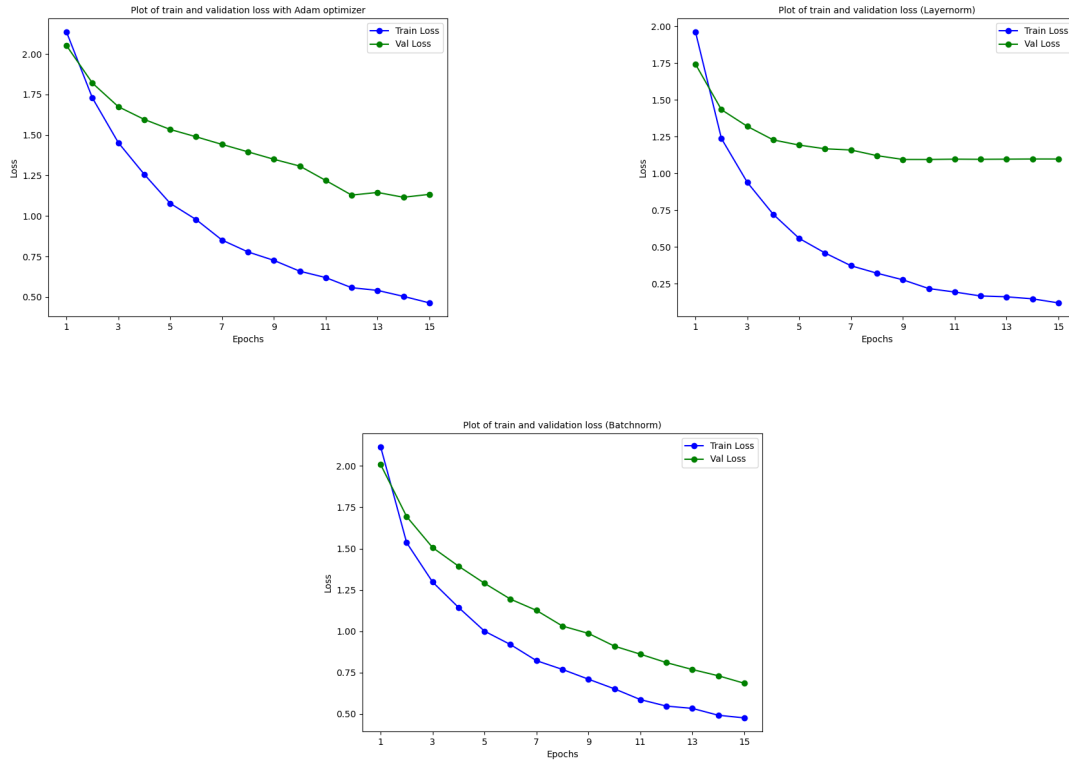


Figure 7: Plot of Training and Validation loss achieved with different norms applied (Note: First plot with Adam and no-norm is same as that of plot with adam optimizer in part(a))

For this part, three models are trained For the first model, trained SGD without any normalization. For the second model used the RMSprop optimizer with Layer norm. For the third model, used Adam optimizer with Batch norm. In the ensembling, I first set the alpha parameters as un-trainable and hence take simple averaging of the three model output logits to make predictions.

Test Accuracy (Average Ensembling) = 66.25%.

(c)-2 Ensembling with optimal linear weighted combination

Now to find the optimal set of parameters $\alpha_1, \alpha_2, \alpha_3$, I make these parameters trainable in the Ensemble Model, freeze the three CNN models and train it only with respect to $\alpha_1, \alpha_2, \alpha_3$ using gradient descent for 15 epochs. The test accuracy obtained was -

Test Accuracy (Optimal Ensembling) = 68.75%.

We can clearly see that the accuracy of the model has increased. This is expected since the optimal weighted combination will find the best weighting parameters to reduce the loss and hence increase the accuracy.

4 CNN and DNN

(i) Computational complexity of a CNN and DNN

To compute the computational complexity of CNN, assume an input size of $C_1 \times H \times W$, let the kernel size be $K \times K$ and the number of output channels be C_2 . The number of multiplications to generate the output can be determined as follows -

To generate one output in one feature map, we need to put the kernel at a specific position on the input signal. The number of multiplications will be - $K \times K \times C_1$. This has to be done for each position of the output feature map and for each output channel. If $C_2 \times H_2 \times W_2$ is the size of the output, then total number of multiplications done will be -

$$= C_1 \times C_2 \times K \times K \times H_2 \times W_2$$

To compute the computational requirements of the **pooling layer** not that pooling layer doesn't do any multiplication. It either does comparison in case of **maxpooling** or averaging in case of **avgpooling**. Hence, the computational complexity of this layer is significantly less. The number of comparisons done is -

$$= C \times H_2 \times W_2 \times K \times K$$

The non-linearity layer does element by element operation. So the non-linearity layers computational complexity is -

$$= C_1 \times H_1 \times W_1 = C_2 \times H_2 \times W_2$$

The equality above because the non-linearity layer maps the input to same sized output.

However, multiplication operation dominates the computational requirement. Hence the order of computation is $O(C_1 C_2 K^2 H_2 W_2)$

Comparison with DNN's computational complexity

CNNs have much lesser computational complexity than DNNs because of sparse connections. This can be illustrated as follows. Assume DNN input and output to be same, we know every neuron in a DNN output is connected with every neuron in the input. So if the output size is $C_2 \times H_2 \times W_2$ and the input size is $C_1 \times H_1 \times W_1$, the total number of multiplications required in forward pass is -

$$= C_1 \times H_1 \times W_1 \times C_2 \times H_2 \times W_2$$

which is significantly higher than $C_1 \times C_2 \times K \times K \times H_2 \times W_2$ for CNNs as $K \ll H_1, W_1$.

Memory requirement of CNN and DNN

Here, by memory requirement, I am assuming the amount of memory required to store the intermediate outputs for tracking and computing gradients while doing forward and backward pass. This is proportional to number of parameters. Since a DNN has high number of parameters compared to CNN, the memory requirement of CNN is much lesser compared to DNN.

Please not in all above discussion, I have assumed same input and output size for CNN and DNN. The only difference being in the format of the input and output, which are 2-D in case of CNN and flattened in case of DNN.

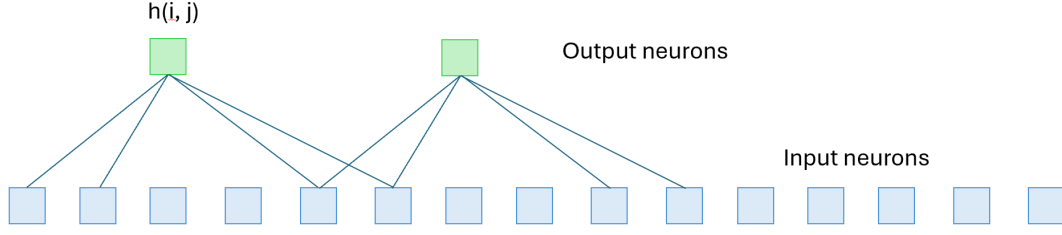


Figure 8: Realization of a Convolution layer from DNN layer ($h(i, j)$ is the output neuron receiving inputs from local regions in the input which have been flattened

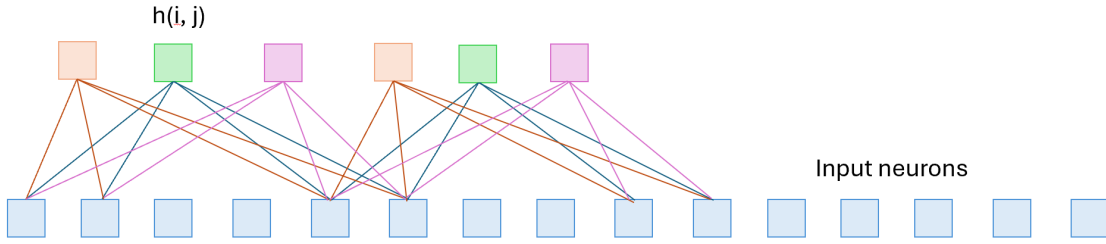


Figure 9: Realizing Multiple kernel operations in CNNs with the use of different weights

(ii) To realize a CNN layer using feedforward layer

We need to show how we can realize convolution, pooling and non-linearity operations done in a convolution layer using the architecture of a feed-forward layer.

The 2-D convolution operation involving input $x(i, j)$ and filter weights $w(m, n)$ can be written as -

$$h(i, j) = \sum_{m=-K}^K \sum_{n=-K}^K w(m, n)x(i + m, j + n)$$

We observe that, to get one value of one output neuron, we need to multiply filter weights to only a few neurons in the input. This can be realized by flattening the input and connecting relevant neurons in the input to the output as dictated by the equation above unlike connecting every neuron in the input to every neuron at the output. This leads to **sparse** connections in the layers as shown in [Figure 8](#)

Now, since, the weights of the filter do not change for different locations of the input. This imply that we will share the weights in the corresponding sparse feed forward counterpart. hence the weights for h_{11} and h_{12} in the [Figure 8](#) the weights are same. This explains **weight sharing** in the CNNs.

However, one might think that this **sparse connection** and **weight sharing** lead to very less number of parameters, so how do we capture the essential features of the input signal as we move forward. Well this is where we use multiple such weights or kernels each sparsely connected with the input with weights sharing for all input locations. This is shown in the ??

To realize a pooling layer, we can use the same logic as above but instead of having to multiply weights, we can use local comparison. This can be thought as the varying weights as we move to different places in the input. The weight corresponding to the pooled element is set to 1 and remaining zero for min or max pooling and for average pooling, we can set all the weights to equal.

To realize a non-linearity layer, we can just connect one element of the input to the same element of the output as non-linearity is an element by element operation.

(ii) To realize a DNN layer using CNN

This is rather simple to do. Since, in DNNs, every neuron at the output is connected to every neuron in the input. We can simply take a kernel whose size is same as the size of the input. However, this will generate only one output neuron as we can't slide the kernel. So we need to have as many kernels as the number of output neurons to realize the DNN.